

AWS - Serverless Architecture & Cloud Automation

Test Name - Serverless architecture

Created by - Saleem Shaikh

Batch - 16A

Submitted Date - June 07 2026

Skill Test Github Link - <https://github.com/saleeminfodevops/herovired.git>

I have picked up the following assignments:

- 1) Assignment 1: Automated Instance Management Using AWS Lambda and Boto3
- 2) Assignment 2: Automated S3 Bucket Cleanup Using AWS Lambda and Boto3
- 3) Assignment 6: Monitor and Alert High AWS Billing Using AWS Lambda, Boto3, and SNS
- 4) Assignment 19: Load Balancer Health Checker
- 5) Assignment 12: Auto-Scale EC2 Instances Based on Load Using AWS Lambda, Boto3, and SNS

1) Assignment 1: Automated Instance Management Using AWS Lambda and Boto3

Objective: In this assignment, you will gain hands-on experience with AWS Lambda and Boto3, Amazon's SDK for Python. You will create a Lambda function that will automatically manage EC2 instances based on their tags.

Task: You're tasked to automate the stopping and starting of EC2 instances based on tags. Specifically:

1. EC2 Setup:

- Navigate to the EC2 dashboard and create two new t2.micro instances (or any other available free-tier type).
- Tag the first instance with a key `Action` and value `Auto-Stop`.
- Tag the second instance with a key `Action` and value `Auto-Start`.

```
testing@inblrlt-002:~/study/assignment$ aws ec2 describe-instances --query "Reservations[*].Instances[*].{PublicAddress:PublicIpAddress,Platform:PlatformDetails,State:State.Name,Name:Tags[?Key=='Name']|[0].Value,Action:Tags[?Key=='Action']|[0].Value}"
```

DescribeInstances						
Action	InstanceID	Name	Platform	PrivateAddress	PublicAddress	State
None	i-037f542b93fe8accd	frontend-01	Linux/UNIX	172.31.12.201	None	stopped
None	i-0400124bba6643b1a	frontend-02	Linux/UNIX	172.31.13.171	None	stopped
Auto-Stop	i-011facb3f85dcfcde	dev-server	Linux/UNIX	172.31.8.179	43.204.230.177	running
Auto-Start	i-03cea284f141c9cba	prod-server	Linux/UNIX	172.31.15.137	None	stopped

```
testing@inblrlt-002:~/study/assignment$
```

2. Lambda IAM Role:

- In the IAM dashboard, create a new role for Lambda.
- Attach the `AmazonEC2FullAccess` policy to this role. (Note: In a real-world scenario, you would want to limit permissions for better security.)
 - Created an inline policy with limited permissions (describe/stop/start).

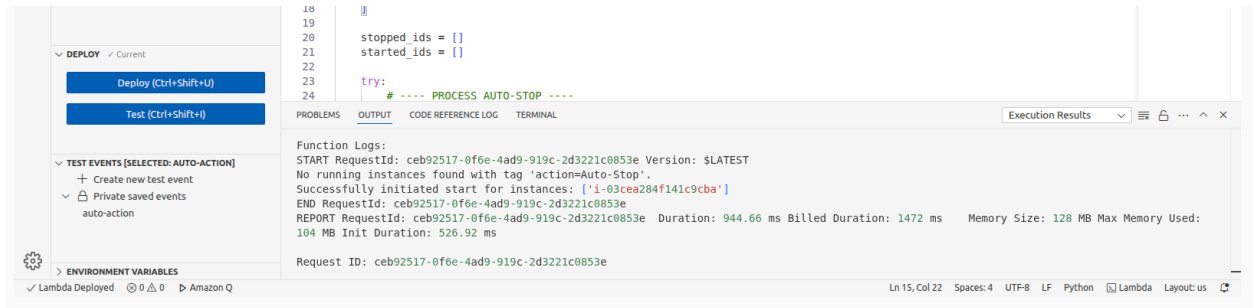
```

assignment > {} start-stop-iam-policy > [ ] Statement
1  {
2      "Version": "2012-10-17",
3      "Statement": [
4          {
5              "Effect": "Allow",
6              "Action": [
7                  "ec2:StartInstances",
8                  "ec2:StopInstances"
9              ],
10             "Resource": "arn:aws:ec2:*:*:instance/*"
11         },
12         {
13             "Effect": "Allow",
14             "Action": "ec2:DescribeInstances",
15             "Resource": "*"
16         }
17     ]
18 }

```

3. Lambda Function:

- Navigate to the Lambda dashboard and create a new function.
- Choose Python 3.x as the runtime.
- Assign the IAM role created in the previous step.
- Write the Boto3 Python script to:
 1. Initialize a boto3 EC2 client.
 2. Describe instances with `Auto-Stop` and `Auto-Start` tags.
 3. Stop the `Auto-Stop` instances and start the `Auto-Start` instances.
 4. Print instance IDs that were affected for logging purposes.



4. Manual Invocation:

- After saving your function, manually trigger it.
- Go to the EC2 dashboard and confirm that the instances' states have changed according to their tags.

```

testing@inblrlt-002: ~$ aws ec2 describe-instances --query "Reservations[*].Instances[*].{PublicAddress:PublicIpAddress,PrivateAddress:PrivateIpAddress,InstanceId:InstanceId,Platform:Platform,State:State,Name:Name,Tags:[?Key=='Name']|[0].Value,Action:Tags[?Key=='Action']|[0].Value}" --output table
DescribeInstances
+-----+-----+-----+-----+-----+-----+-----+
| Action | InstanceID | Name | Platform | PrivateAddress | PublicAddress | State |
+-----+-----+-----+-----+-----+-----+-----+
| None | i-037f542b93fe8accd | frontend-01 | Linux/UNIX | 172.31.12.201 | None | stopped |
| None | i-0400124bba6643b1a | frontend-02 | Linux/UNIX | 172.31.13.171 | None | stopped |
| Auto-Stop | i-011facb3f85dcfcde | dev-server | Linux/UNIX | 172.31.8.179 | 65.0.173.7 | running |
| Auto-Start | i-03cea284f141c9cba | prod-server | Linux/UNIX | 172.31.15.137 | None | stopped |
+-----+-----+-----+-----+-----+-----+-----+

testing@inblrlt-002: ~$ aws lambda invoke \
--function-name arn:aws:lambda:ap-south-1:135808938060:function:stop-start-ec2 \
--cli-binary-format raw-in-base64-out \
--payload '{ "key": "value" }' \
response.json
{"statusCode": 200,
"ExecutedVersion": "$LATEST"}

testing@inblrlt-002: ~$ aws ec2 describe-instances --query "Reservations[*].Instances[*].{PublicAddress:PublicIpAddress,PrivateAddress:PrivateIpAddress,InstanceId:InstanceId,Platform:Platform,State:State,Name:Name,Tags:[?Key=='Name']|[0].Value,Action:Tags[?Key=='Action']|[0].Value}" --output table
DescribeInstances
+-----+-----+-----+-----+-----+-----+-----+
| Action | InstanceID | Name | Platform | PrivateAddress | PublicAddress | State |
+-----+-----+-----+-----+-----+-----+-----+
| None | i-037f542b93fe8accd | frontend-01 | Linux/UNIX | 172.31.12.201 | None | stopped |
| None | i-0400124bba6643b1a | frontend-02 | Linux/UNIX | 172.31.13.171 | None | stopped |
| Auto-Stop | i-011facb3f85dcfcde | dev-server | Linux/UNIX | 172.31.8.179 | 65.0.173.7 | stopping |
| Auto-Start | i-03cea284f141c9cba | prod-server | Linux/UNIX | 172.31.15.137 | 15.207.71.115 | running |
+-----+-----+-----+-----+-----+-----+-----+

testing@inblrlt-002: ~$ aws ec2 describe-instances --query "Reservations[*].Instances[*].{PublicAddress:PublicIpAddress,PrivateAddress:PrivateIpAddress,InstanceId:InstanceId,Platform:Platform,State:State,Name:Name,Tags:[?Key=='Name']|[0].Value,Action:Tags[?Key=='Action']|[0].Value}" --output table
DescribeInstances
+-----+-----+-----+-----+-----+-----+-----+
| Action | InstanceID | Name | Platform | PrivateAddress | PublicAddress | State |
+-----+-----+-----+-----+-----+-----+-----+
| None | i-037f542b93fe8accd | frontend-01 | Linux/UNIX | 172.31.12.201 | None | stopped |
| None | i-0400124bba6643b1a | frontend-02 | Linux/UNIX | 172.31.13.171 | None | stopped |
| Auto-Stop | i-011facb3f85dcfcde | dev-server | Linux/UNIX | 172.31.8.179 | None | stopped |
| Auto-Start | i-03cea284f141c9cba | prod-server | Linux/UNIX | 172.31.15.137 | 15.207.71.115 | running |
+-----+-----+-----+-----+-----+-----+-----+

testing@inblrlt-002: ~$ ls
Desktop Documents Downloads fbs index.html kubectll kubectll.sha256 Music Pictures Public response.json skil snap study Templates Videos
testing@inblrlt-002: ~$ cat response.json
{"statusCode": 200, "body": "{\"StoppedInstances\": [\"i-011facb3f85dcfcde\"], \"StartedInstances\": [\"i-03cea284f141c9cba\"]}"}
testing@inblrlt-002: ~$

```

Assignment 2: Automated S3 Bucket Cleanup Using AWS Lambda and Boto3

Objective: To gain experience with AWS Lambda and Boto3 by creating a Lambda function that will automatically clean up old files in an S3 bucket.

Task: Automate the deletion of files older than 30 days in a specific S3 bucket.

Instructions:

1. S3 Setup:

- Navigate to the S3 dashboard and create a new bucket.
- Upload multiple files to this bucket, ensuring that some files are older than 30 days (you may need to adjust your system's date temporarily for this or use old files).

ftg-s3-01

< **ftg-s3-01** [Info](#)

[Objects](#) [Metadata](#) [Properties](#) [Permissions](#) [Metrics](#) [Management](#) [File systems - new](#) [Access Points](#)

Objects (4) [Refresh](#) [Copy S3 URI](#) [Copy URL](#) [Download](#) [Open L2](#) [Delete](#)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, see [Access control](#).

☐	Name	Type	Last modified	Size
☐	aws_security.pptx	pptx	April 18, 2026, 21:04:12 (UTC+05:30)	
☐	certs/	Folder	-	
☐	photos/	Folder	-	
☐	video/	Folder	-	

2. Lambda IAM Role:

- In the IAM dashboard, create a new role for Lambda.
- Attach the `AmazonS3FullAccess` policy to this role. (Note: For enhanced security in real-world scenarios, use more restrictive permissions.)

s3-onlydata-management-policy [Info](#)

Policy details

Type	Creation time
Customer managed	May 31, 2026, 10:37 (UTC+05:30)

Permissions

Entities attached

Tags (1)

Policy versions (2)

Last Acc

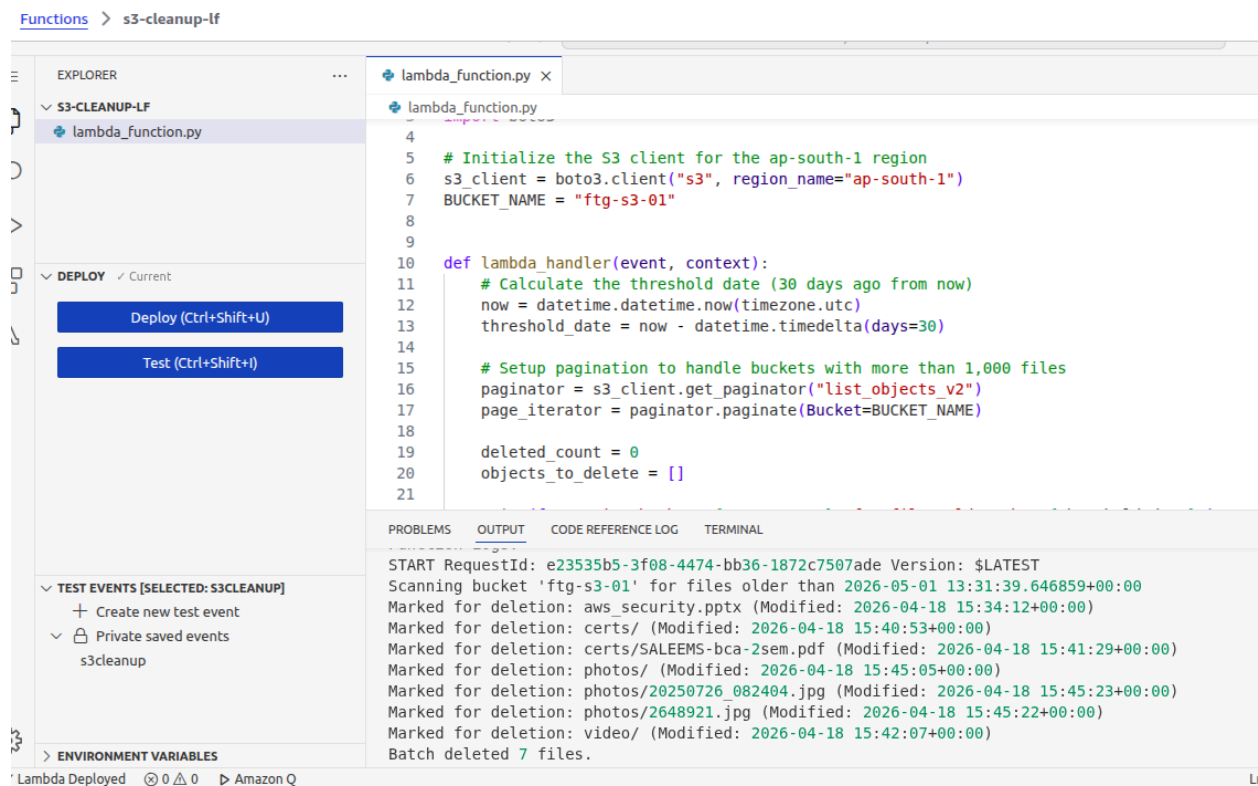
Permissions defined in this policy [Info](#)

Permissions defined in this policy document specify which actions are allowed or denied. To define

```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Action": [
7         "s3:ListBucket",
8         "s3:DeleteObject"
9       ],
10      "Resource": [
11        "arn:aws:s3:::ftg-s3-01",
12        "arn:aws:s3:::ftg-s3-01/*"
13      ]
14    }
15  ]
16 }
```

3. Lambda Function:

- Navigate to the Lambda dashboard and create a new function.
- Choose Python 3.x as the runtime.
- Assign the IAM role created in the previous step.
- Write the Boto3 Python script to:
 1. Initialize a boto3 S3 client.
 2. List objects in the specified bucket.
 3. Delete objects older than 30 days.
 4. Print the names of deleted objects for logging purposes.



```
4
5 # Initialize the S3 client for the ap-south-1 region
6 s3_client = boto3.client("s3", region_name="ap-south-1")
7 BUCKET_NAME = "ftg-s3-01"
8
9
10 def lambda_handler(event, context):
11     # Calculate the threshold date (30 days ago from now)
12     now = datetime.datetime.now(timezone.utc)
13     threshold_date = now - datetime.timedelta(days=30)
14
15     # Setup pagination to handle buckets with more than 1,000 files
16     paginator = s3_client.get_paginator("list_objects_v2")
17     page_iterator = paginator.paginate(Bucket=BUCKET_NAME)
18
19     deleted_count = 0
20     objects_to_delete = []
21
```

START RequestId: e23535b5-3f08-4474-bb36-1872c7507ade Version: \$LATEST
Scanning bucket 'ftg-s3-01' for files older than 2026-05-01 13:31:39.646859+00:00
Marked for deletion: aws_security.pptx (Modified: 2026-04-18 15:34:12+00:00)
Marked for deletion: certs/ (Modified: 2026-04-18 15:40:53+00:00)
Marked for deletion: certs/SALEEMS-bca-2sem.pdf (Modified: 2026-04-18 15:41:29+00:00)
Marked for deletion: photos/ (Modified: 2026-04-18 15:45:05+00:00)
Marked for deletion: photos/20250726_082404.jpg (Modified: 2026-04-18 15:45:23+00:00)
Marked for deletion: photos/2648921.jpg (Modified: 2026-04-18 15:45:22+00:00)
Marked for deletion: video/ (Modified: 2026-04-18 15:42:07+00:00)
Batch deleted 7 files.

4. Manual Invocation:

- After saving your function, manually trigger it.

- Go to the S3 dashboard and confirm that only files newer than 30 days remain.

[zon S3](#) > [Buckets](#) > **ftg-s3-01**

ftg-s3-01 Info

Objects

Metadata

Properties

Permissions

Metrics

Management

File sy

Objects (1)

 Copy S3 URI

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all obje

 Find objects by prefix

☐	Name	▲	Type	▼	Last modifi
☐	 photos/		Folder		-

Assignment 6: Monitor and Alert High AWS Billing Using AWS Lambda, Boto3, and SNS

Objective: Create an automated alerting mechanism for when your AWS billing exceeds a certain threshold.

Task: Set up a Lambda function to check your AWS billing amount daily, and if it exceeds a specified threshold, send an alert via SNS.

Instructions:

1. SNS Setup:

- Navigate to the SNS dashboard and create a new topic.

The screenshot shows the AWS SNS console for a topic named 'monitor-billing'. The 'Details' tab is active, displaying the topic's name, ARN, display name, and type (Standard). Below this, the 'Subscriptions' tab is selected, showing a list of subscriptions. One subscription is listed with ID '6081a28e-55e8-47a1-990a-9771fa226474', endpoint 'falcontechguru@gmail.com', status 'Confirmed', and protocol 'EMAIL'. The console also shows tabs for 'Access policy', 'Delivery policy (HTTP/S)', 'Delivery status logging', 'Encryption', 'Tags', and 'Integrations'.

monitor-billing			
Details			
Name	ARN	Display name	Type
monitor-billing	arn:aws:sns:ap-south-1:135808938060:monitor-billing	monitor-billing	Standard
Topic owner: 135808938060			
Subscriptions (1)			
ID	Endpoint	Status	Protocol
6081a28e-55e8-47a1-990a-9771fa226474	falcontechguru@gmail.com	Confirmed	EMAIL

- Subscribe your email to this topic.

The screenshot shows the AWS SNS console for a specific subscription. The 'Details' tab is active, displaying the subscription's ARN, endpoint, topic, status (Confirmed), and protocol (EMAIL). Below this, the 'Subscription filter policy' tab is selected, showing that no filter policy is configured for this subscription.

Subscription: 6081a28e-55e8-47a1-990a-9771fa226474	
Details	
ARN	Status
arn:aws:sns:ap-south-1:135808938060:monitor-billing:6081a28e-55e8-47a1-990a-9771fa226474	Confirmed
Endpoint	Protocol
falcontechguru@gmail.com	EMAIL
Topic	
monitor-billing	
Subscription filter policy	
No filter policy configured for this subscription	

Enabled Receive AWS Free Tier alerts

Billing preferences [Info](#)

Invoice delivery preferences [Info](#)

PDF invoices delivery by email

Deactivated

Edit

Alert preferences [Info](#)

AWS Free Tier alerts

Delivered to falcontechguru@gmail.com

CloudWatch billing alerts

Delivered

Edit

2. Lambda IAM Role:

- In the IAM dashboard, create a new role for Lambda.
- Attach policies that allow reading CloudWatch metrics and sending SNS notifications.

bill-monitor-policy [Info](#)

Policy details

Type	Creation time	Edited time
Customer managed	May 31, 2026, 20:08 (UTC+05:30)	May 31, 2026, 20:08 (UTC+05:30)

[Permissions](#) | [Entities attached](#) | [Tags \(1\)](#) | [Policy versions \(1\)](#) | [Last Accessed](#)

Permissions defined in this policy [Info](#)

Permissions defined in this policy document specify which actions are allowed or denied. To define permissions for an IAM identity (user, user group, or role),

Allow (2 of 469 services)

Service	Access level	Resource	Request condition
CloudWatch	Limited: List, Read	All resources	None
SNS	Limited: List, Read, Write	All resources	None

3. Lambda Function:

- Navigate to the Lambda dashboard and create a new function.
- Choose Python 3.x as the runtime.
- Assign the IAM role created in the previous step.
- Write the Boto3 Python script to:
 1. Initialize boto3 clients for CloudWatch and SNS.
 2. Retrieve the AWS billing metric from CloudWatch.
 3. Compare the billing amount with a threshold (e.g., \$50).
 4. If the billing exceeds the threshold, send an SNS notification.
 5. Print messages for logging purposes.

> monitor-billing-lf

Current

Deploy (Ctrl+Shift+U)

Test (Ctrl+Shift+I)

VENTS [SELECTED: MONITOR-BILL]

Create new test event

Private saved events

monitor-bill

ENVIRONMENT VARIABLES

oyed 0 0 0 Amazon Q

```
23 response = cloudwatch.get_metric_statistics(  
24     Namespace='AWS/Billing',  
25     MetricName='EstimatedCharges',  
26     Dimensions=[  
27         {  
28             'Name': 'Currency',  
29             'Value': 'USD'  
30         }  
31     ],  
32     StartTime=start_time,  
33     EndTime=end_time,  
34     Period=21600, # 6 hours  
35     Statistics=['Maximum']  
36 )  
37  
38 datapoints = response.get('Datapoints', [])  
39
```

PROBLEMS

OUTPUT

CODE REFERENCE LOG

TERMINAL

Response:

```
{  
  "statusCode": 200,  
  "body": "No billing data available yet."  
}
```

The area below shows the last 4 KB of the execution log.

Function Logs:

Ln 84

4. Event Source (Bonus):

- Attach an event source, like Amazon CloudWatch Events, to trigger the Lambda function daily.

The screenshot displays the AWS Lambda console interface for the function **monitor-billing-lf**. At the top, there's a 'Function overview' section with tabs for 'Diagram' and 'Template'. Below this, the function's layers are listed as '(0)'. A diagram shows the function connected to an 'EventBridge (CloudWatch Events)' source. A '+ Add destination' button is visible on the right. The main navigation bar includes 'Code', 'Test', 'Monitor', 'Configuration' (selected), 'Aliases', and 'Versions'. The left sidebar under 'Configuration' lists 'General configuration', 'Triggers' (selected), 'Permissions', 'Destinations', 'Function URL', 'Environment variables', 'Tags', and 'VPC'. The 'Triggers (1)' section shows a search bar and a table with one trigger:

☐	Trigger
☐	EventBridge (CloudWatch Events): billing-monitor arn:aws:events:ap-south-1:135808938060:rule/billing-monitor Rule state: ENABLED ▼ Details Event bus: default IsComplexStatement: No Schedule expression: cron(0 21 * * ? *) Service principal: events.amazonaws.com Statement ID: lambda-f8d0ef32-d0b8-4349-9f08-d6d754363f20

5. Testing:

- Manually trigger the Lambda function or wait for the scheduled event.
- If your billing is over the threshold, you should receive an email alert.

Assignment 19: Load Balancer Health Checker

Objective: Design a Lambda function that checks the health of registered instances behind an Elastic Load Balancer (ELB) and notifies via SNS if any instances are unhealthy.

Instructions:

1. Create a Lambda function.
2. With Boto3, configure the function to:

```
import json
import boto3
from datetime import datetime

elbv2 = boto3.client('elbv2')
sns = boto3.client('sns')

ALB_ARN =
"arn:aws:elasticloadbalancing:ap-south-1:135808938060:loadbalancer/app/ftg-1b/f9c180112d3b5945"
SNS_TOPIC_ARN =
"arn:aws:sns:ap-south-1:135808938060:loadbalancer-healthcheck-topic"

def lambda_handler(event, context):

    unhealthy_targets = []

    try:
        # Get all target groups attached to ALB
        target_groups = elbv2.describe_target_groups(
            LoadBalancerArn=ALB_ARN
        )['TargetGroups']

        for tg in target_groups:

            tg_name = tg['TargetGroupName']
            tg_arn = tg['TargetGroupArn']

            target_health = elbv2.describe_target_health(
```

```

        TargetGroupArn=tg_arn
    )

    for target in target_health['TargetHealthDescriptions']:

        target_id = target['Target']['Id']
        target_port = target['Target']['Port']

        state = target['TargetHealth']['State']
        reason = target['TargetHealth'].get('Reason', 'N/A')
        description = target['TargetHealth'].get('Description',
'N/A')

        if state != 'healthy':
            unhealthy_targets.append({
                "TargetGroup": tg_name,
                "InstanceId": target_id,
                "Port": target_port,
                "State": state,
                "Reason": reason,
                "Description": description
            })

    if unhealthy_targets:

        message = {
            "Timestamp": datetime.utcnow().isoformat(),
            "LoadBalancerArn": ALB_ARN,
            "UnhealthyTargets": unhealthy_targets
        }

        sns.publish(
            TopicArn=SNS_TOPIC_ARN,
            Subject="ALB Health Check Alert - Unhealthy Targets
Detected",
            Message=json.dumps(message, indent=4)
        )

    return {
        "statusCode": 200,

```

```

        "message": f"{len(unhealthy_targets)} unhealthy targets
found and SNS notification sent."
    }

    return {
        "statusCode": 200,
        "message": "All targets are healthy."
    }

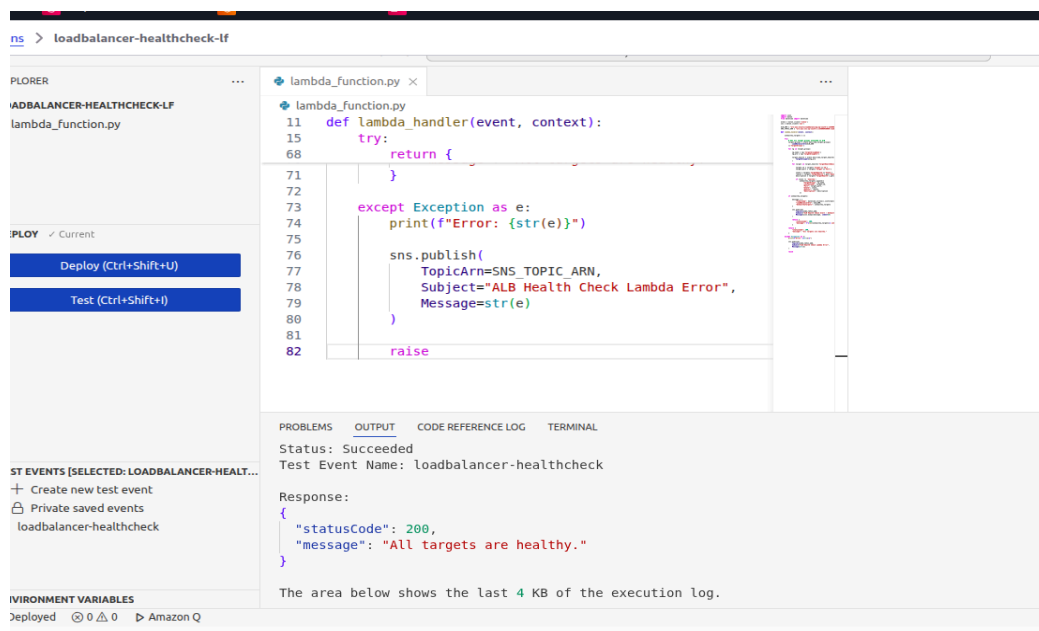
except Exception as e:
    print(f"Error: {str(e)}")

    sns.publish(
        TopicArn=SNS_TOPIC_ARN,
        Subject="ALB Health Check Lambda Error",
        Message=str(e)
    )

    raise

```

1. Check the health of registered instances behind a given ELB.



Create Topic and register email

loadbalancer-healthcheck-topic

Details

Name loadbalancer-healthcheck-topic	ARN arn:aws:sns:ap-south-1:135808938060:loadbalancer-healthcheck-topic	Display name loadbalancer-healthcheck-topic	Type Standard
---	--	---	-------------------------

Topic owner
135808938060

Subscriptions | Access policy | Delivery policy (HTTP/S) | Delivery status logging | Encryption | Tags | Integrations

Subscriptions (1)

ID	Endpoint	Status	Protocol
5ca5bf4e-64aa-44fc-be99-03563978072f	falcontechguru@gmail.com	Confirmed	EMAIL

Create an EventBridge to trigger the Lambda function every 10 minutes

loadbalancer-healthcheck-lf

Layers (0)

EventBridge (CloudWatch Events)

+ Add trigger

+ Add destination

Configuration

- General configuration
- Triggers**
- Permissions
- Destinations
- Function URL
- Environment variables
- Tags
- VPC
- RDS databases

Triggers (1) Info

Search for triggers

☐ Trigger

EventBridge (CloudWatch Events): loadbalancer-healthcheck-eventbridge
arn:aws:events:ap-south-1:135808938060:rule/loadbalancer-healthcheck-eventbridge
Rule state: **ENABLED**

Details

Description: **loadbalancer-healthcheck-eventbridge**
Event bus: **default**
IsComplexStatement: **No**
Schedule expression: **cron(0/10 * * * ? *)**
Service principal: **events.amazonaws.com**
Statement ID: **lambda-00dd4f14-4c9d-41e8-9450-7cdd1df08b8b**

2. If any instances are found to be unhealthy, publish a detailed message to an SNS topic.

Details
arn:aws:elasticloadbalancing:ap-south-1:135808938060:targetgroup/ftg-tg/e27b3cf65ef6451d

Target type Instance	Protocol : Port HTTP: 80	Protocol version HTTP1	VPC vpc-04db9c37f3ade1fdc
IP address type IPv4	Load balancer ftg-lb		

1	0	1	0	0	0
Total targets	Healthy	Unhealthy	Unused	Initial	Draining
	0 Anomalous				

► **Distribution of targets by Availability Zone (AZ)**
Select values in this table to see corresponding filters applied to the Registered targets table below.

Targets | Monitoring | Health checks | Attributes | Tags

Registered targets (1) [Info](#) [Anomaly mitigation: Not applicable](#) [Deregister](#) [Register targ](#)

Target groups route requests to individual registered targets using the protocol and port number specified. Health checks are performed on all registered targets according to the target group's health check settings. Anomaly detect automatically applied to HTTP/HTTPS target groups with at least 3 healthy targets.

Q Filter targets

Instance ID	Name	Port	Zone	Health status	Health status details	Administrativ...	Override details	Launch...
i-037f542b93fe8accd	frontend-01	80	ap-south-1b (a...	Unhealthy	Health checks failed	No override	No override is cur...	June 6, 20...

3. Set up a CloudWatch event to trigger this Lambda function every 5 minutes.

Search mail

1 of 80

ALB Health Check Alert - Unhealthy Targets Detected [Inbox x](#)

loadbalancer-healthcheck-topic <no-reply@sns.amazonaws.com> 10:15 PM (0 minutes ago) ☆ ☺

to me

```
{
  "Timestamp": "2026-06-07T16:45:47.854434",
  "LoadBalancerArn": "arn:aws:elasticloadbalancing:ap-south-1:135808938060:loadbalancer/app/ftg-lb/f9c180112d3b5945",
  "UnhealthyTargets": [
    {
      "TargetGroup": "ftg-tg",
      "InstanceId": "i-037f542b93fe8accd",
      "Port": 80,
      "State": "unhealthy",
      "Reason": "TargetFailedHealthChecks",
      "Description": "Health checks failed"
    }
  ]
}
```

--

If you wish to stop receiving notifications from this topic, please click or visit the link below to unsubscribe:
<https://sns.ap-south-1.amazonaws.com/unsubscribe.htm?SubscriptionArn=arn:aws:sns:ap-south-1:135808938060:loadbalancer-healthcheck-topic:5ca5bf4e-64aa-44fc-be99-03563978072f&Endpoint=falcontechguru@gmail.com>

Please do not reply directly to this email. If you have any questions or comments regarding this email, please contact us at <https://aws.amazon.com/support>

Assignment 12: Auto-Scale EC2 Instances Based on Load Using AWS Lambda, Boto3, and SNS

Objective: Automatically scale up or down the number of EC2 instances based on network load.

Task: Set up a Lambda function that checks the network load on an ELB and scales the number of EC2 instances accordingly.

Instructions:

1. SNS Setup:

- Create a new SNS topic.
- Subscribe to the topic.

auto-scale-topic [Edit](#)

Details

Name  auto-scale-topic	ARN  arn:aws:sns:ap-south-1:135808938060:auto-scale-topic	Display name auto-scale-topic	Type Standard
Topic owner 135808938060			

[Subscriptions](#) | [Access policy](#) | [Delivery policy \(HTTP/S\)](#) | [Delivery status logging](#) | [Encryption](#) | [Tags](#) | [Integrations](#)

Subscriptions (1) [Edit](#) [Delete](#) [Request confirmation](#) [Confirm sub](#)

ID	Endpoint	Status	Protocol
 635696e9-9e87-4bac-9fcf-55d2b71d0d52	falcontechguru@gmail.com	 Confirmed	EMAIL

2. Lambda IAM Role:

- Create a role with permissions to read ELB metrics, start/terminate EC2 instances, and send SNS notifications.

elb-ec2autoscaling-policy

Info

Policy details

Type

Customer managed

Creation time

May 31, 2026, 21:32 (UTC+05:30)

Edited time

May 31, 2026, 21:32 (UTC+05:30)

Permissions

Entities attached

Tags (1)

Policy versions (1)

Last Accessed

Permissions defined in this policy

Info

Permissions defined in this policy document specify which actions are allowed or denied. To define permissions for an IAM identity (user, user group, or role), att

Q Search

Allow (5 of 469 services)

Service	Access level	Resource	Request condition
CloudWatch	Limited: List, Read	All resources	None
EC2	Limited: Write	All resources	None
ELB	Full: List Limited: Read	All resources	None
ELB v2	Limited: List	All resources	None
SNS	Limited: Write	All resources	None

elb-ec2autoscaling-role

Info

Allows Lambda functions to call AWS services on your behalf.

Summary

Creation date

May 31, 2026, 21:33 (UTC+05:30)

Last activity

-

ARN

 arn:aws:iam::135808938

Maximum session duration

1 hour

Permissions

Trust relationships

Tags (1)

Last Accessed

Revoke sessions

Permissions policies (1)

Info

You can attach up to 10 managed policies.

Q Search

Filter by Type

All types

☐

Policy name

▲

Type

☐

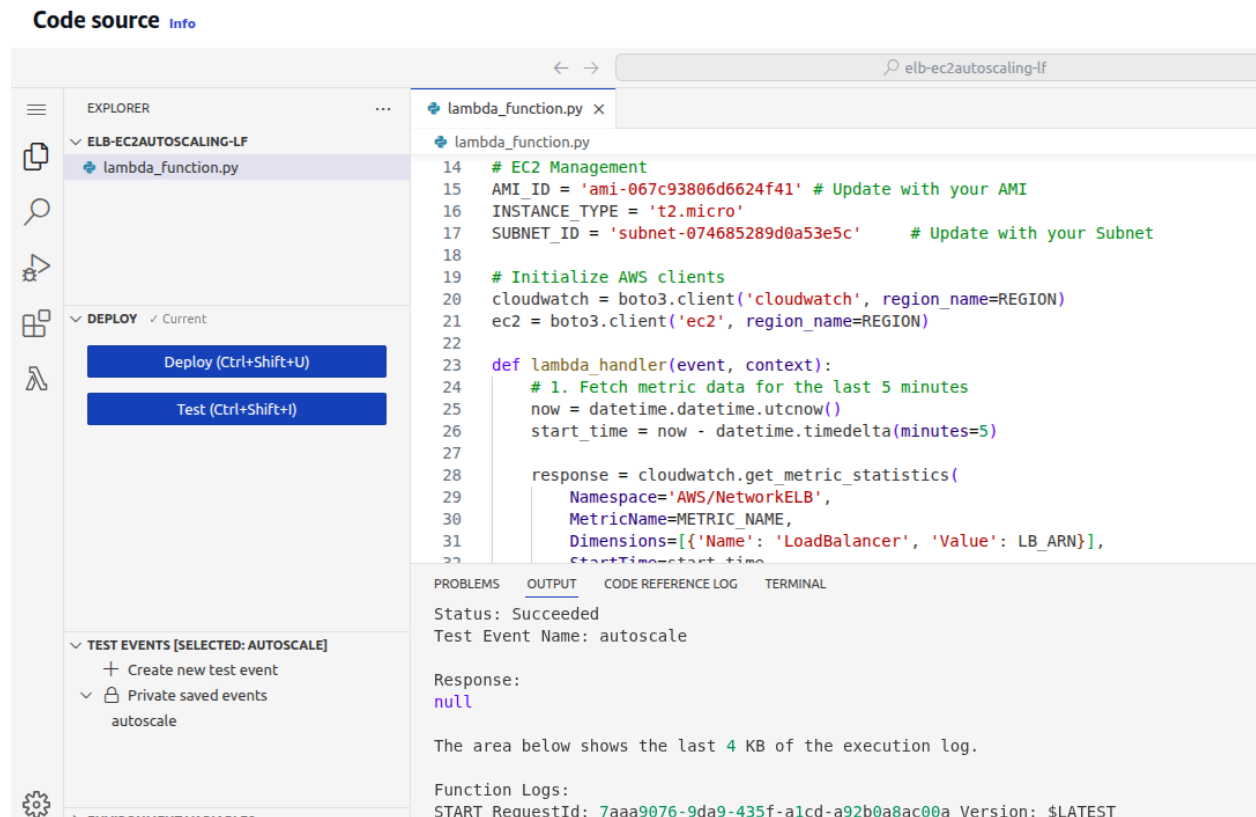
 [elb-ec2autoscaling-policy](#)

Customer managed

3. Lambda Function:

- Create a function with the above IAM role.
- Use Boto3 to:
 1. Fetch the network load metric from ELB.
 2. If the load is above a high threshold (e.g., 80% for 5 minutes), launch a new EC2 instance.
 3. If the load is below a low threshold (e.g., 20% for 5 minutes), terminate an EC2 instance.
 4. Notify via SNS about the scaling action taken.

Code source [Info](#)



```
14 # EC2 Management
15 AMI_ID = 'ami-067c93806d6624f41' # Update with your AMI
16 INSTANCE_TYPE = 't2.micro'
17 SUBNET_ID = 'subnet-074685289d0a53e5c' # Update with your Subnet
18
19 # Initialize AWS clients
20 cloudwatch = boto3.client('cloudwatch', region_name=REGION)
21 ec2 = boto3.client('ec2', region_name=REGION)
22
23 def lambda_handler(event, context):
24     # 1. Fetch metric data for the last 5 minutes
25     now = datetime.datetime.utcnow()
26     start_time = now - datetime.timedelta(minutes=5)
27
28     response = cloudwatch.get_metric_statistics(
29         Namespace='AWS/NetworkELB',
30         MetricName=METRIC_NAME,
31         Dimensions=[{'Name': 'LoadBalancer', 'Value': LB_ARN}],
32         StartTime=start_time,
```

PROBLEMS **OUTPUT** CODE REFERENCE LOG TERMINAL

Status: Succeeded
Test Event Name: autoscale

Response:
null

The area below shows the last 4 KB of the execution log.

Function Logs:
START RequestId: 7aaa9076-9da9-435f-a1cd-a92b0a8ac00a Version: \$LATEST

4. CloudWatch Events:

- Schedule your Lambda function to run every 5 minutes.

5. Testing:

- Simulate varying network loads.

- Confirm that EC2 instances are correctly scaled and SNS notifications are received.